

# Beyond AUC, in Plain English

**A companion to the research write-up** — *what each step is, why it exists, and what the numbers actually mean. No statistics background assumed.*

## What this project is, in one paragraph

---

When you buy something online, a computer decides — in a fraction of a second — whether to approve or decline your card. That computer faces two ways to be wrong. It can approve a fraudster (the bank eats the loss), or it can decline a real customer (the store loses the sale, and an embarrassed customer may never come back). Industry studies suggest the second mistake is, in total dollars, the more expensive one: one widely-cited estimate put falsely declined transactions at \$118 billion a year in the U.S., far more than the fraud itself. This project builds a fraud-detection system on 590,540 real online transactions and — this is the point — grades it the way a payments company would have to: in dollars, on future data it has never seen, with every safeguard against accidentally cheating. The claim of the project is not "here is the world's best fraud model." It is: *here is what an honestly evaluated fraud decision system looks like, and here is how much the usual shortcuts flatter the results.*

## The data, and the two mistakes

---

The dataset (published by payments company Vesta for a 2019 competition) contains six months of e-commerce card transactions. Each row is one purchase — its amount, card details, email domain, device information, and hundreds of anonymized signals — plus a label: was this later confirmed as fraud? Only **3.5%** of transactions are fraud, which means a lazy system that approves everything is right 96.5% of the time and still loses every fraudulent dollar. That imbalance is why fraud detection needs its own careful yardsticks.

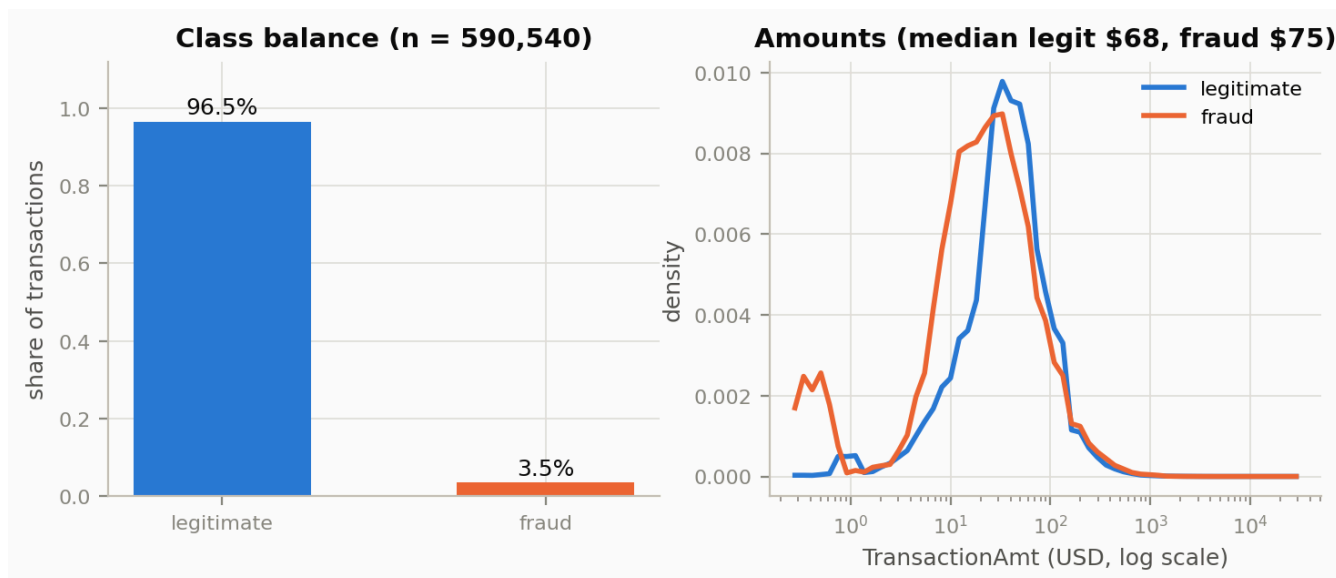


Figure 1 — class balance and amounts: fraud is rare, and fraudulent amounts look a lot like legitimate ones

Notice in the right panel that fraud amounts sit almost on top of legitimate amounts — you cannot spot fraud by price tag alone. The signal lives in *behavior*: how often this card shows up, from what device, at what hours, matching what history.

## How the work was organized: stage gates

The project ran as ten numbered stages, each with a **stage gate**: an explicit checklist that must pass before the next stage may begin. Stage 0's gate, for example, was "the official dataset is present, its digital fingerprint (checksum) matches the original download exactly, and its shape matches the documentation." A gate sounds bureaucratic, but it is the cheapest insurance in data science: if your data is subtly wrong, everything built on it is elaborate nonsense. The gate makes "the data is right" a *verified fact* rather than an assumption.

## Step by step through the pipeline

### EDA — looking before modeling

**EDA (exploratory data analysis)** is the unglamorous step of simply *looking* at the data: how much of it is there, what's missing, how it moves over time. Two findings shaped everything after. First, the fraud rate is not stable — it drifts from 2.5% to over 4% month to month. Second, whole blocks of information are frequently blank (device details exist for only a quarter of transactions). A system built on a snapshot of this world will slowly stop matching it.

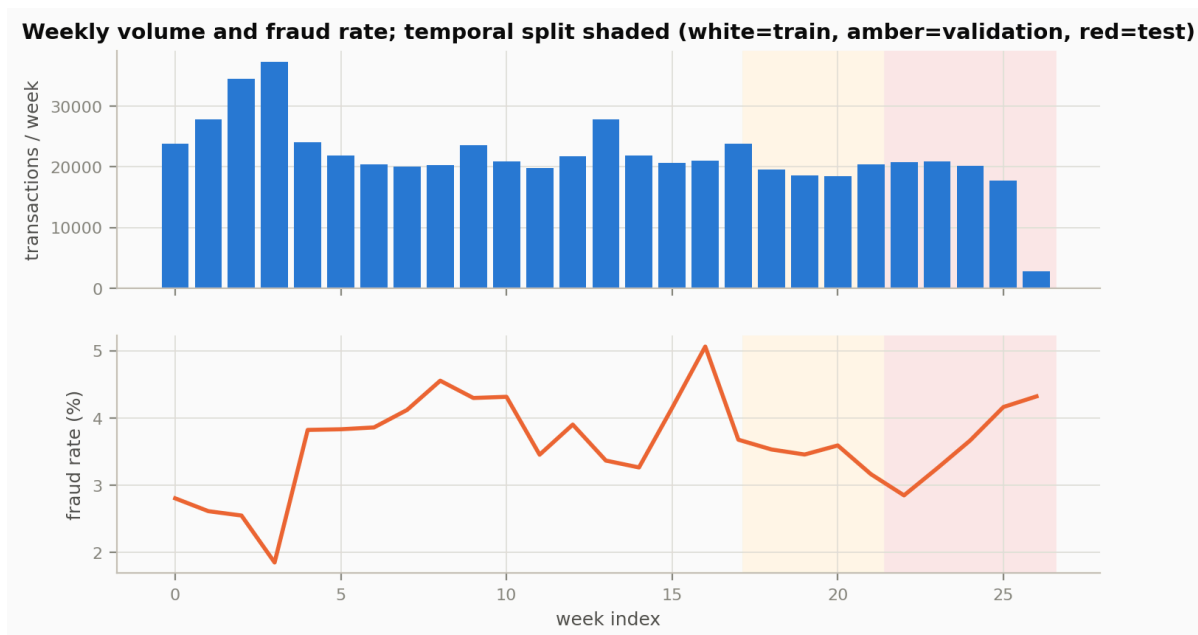


Figure 2 — the six months of data, with the time-based split shaded

## The leakage audit — checking for time machines

**Leakage** is when information from the future sneaks into a model's training, making it look brilliant in the lab and useless in production. Fraud data is full of leakage traps. The classic one here: features like "how many times has this card appeared?" If computed over the *whole* dataset, that count includes appearances that haven't happened yet — a time machine. Every history-based feature in this project is built **causally**: a transaction may only see what happened strictly before it, exactly as a live system would. This isn't left to good intentions; an automated test deletes the "future" rows and re-computes each feature, verifying the answer doesn't change. The audit also explains why the data can't be shuffled for testing (next section) — a lesson learned publicly by the competition's winners, who found that the model's real skill was recognizing *card profiles* it had already seen.

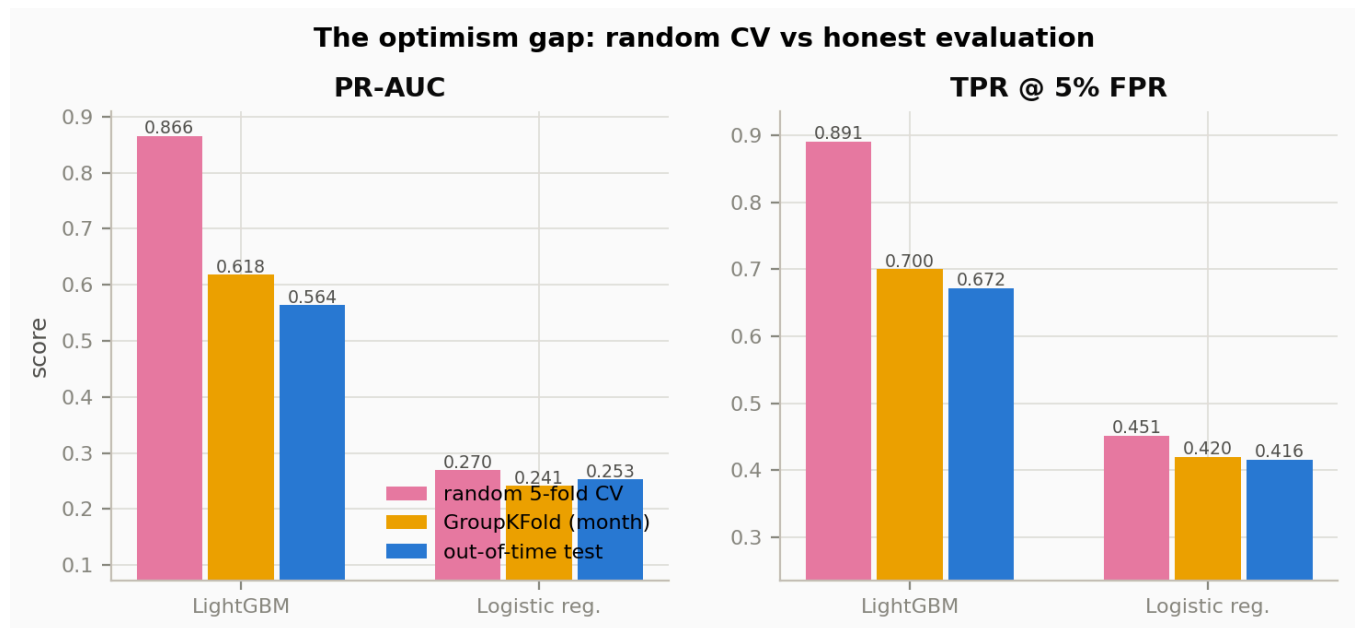
## Baselines and the champion-challenger setup

A **baseline** is the simple thing you must beat before congratulating yourself. Here two models are trained. The **champion** is logistic regression — a straightforward, auditable technique from classical statistics that effectively assigns points for risk factors. The **challenger** is LightGBM — a modern "gradient-boosted trees" method that builds thousands of small decision rules and adds them up. This champion-challenger framing is standard in banks: the simple model keeps the sophisticated one honest.

## The optimism gap — the headline warning

Machine-learning practice commonly grades models by shuffling all the data and testing on random held-out slices (**cross-validation**). With time-ordered fraud data that is grading with the

answer key: the model trains on July and is quizzed on May. Graded that way, the challenger looks spectacular. Graded honestly — trained on months 0–3, tested on the untouched month 5 — the same model with the same settings scores dramatically lower: one key accuracy measure (PR-AUC) falls from **0.87 to 0.56**. That difference is the **optimism gap**. Tellingly, the simple champion barely inflates at all — the gap comes from the powerful model's ability to *memorize* card behavior across a shuffled split. Think grade inflation: same student, same exam, but one grading scheme lets them peek.



### Calibration — making the scores mean what they say

Models output a score between 0 and 1, but out of the box the score is *not* an honest probability (the training procedure deliberately over-weights the rare fraud cases, which warps the scale). **Calibration** fixes the scale. The test of honesty: among all transactions the model scores "30%", almost exactly 30% should actually be fraud — like a weather forecaster whose "30% chance of rain" days really do rain 3 times in 10. Two standard fixes were tried on a held-out month: **Platt scaling** (fits one smooth S-shaped correction curve) and **isotonic regression** (fits a flexible staircase that can bend anywhere, as long as it never goes backwards — that's what "isotonic" means). Isotonic won. The scoreboard uses **ECE (expected calibration error)** — roughly, "on average, when the model says X%, how far off is reality?" — which improved from 1.5% to 0.4%, and the **Brier score**, a combined measure of accuracy-and-honesty where lower is better.

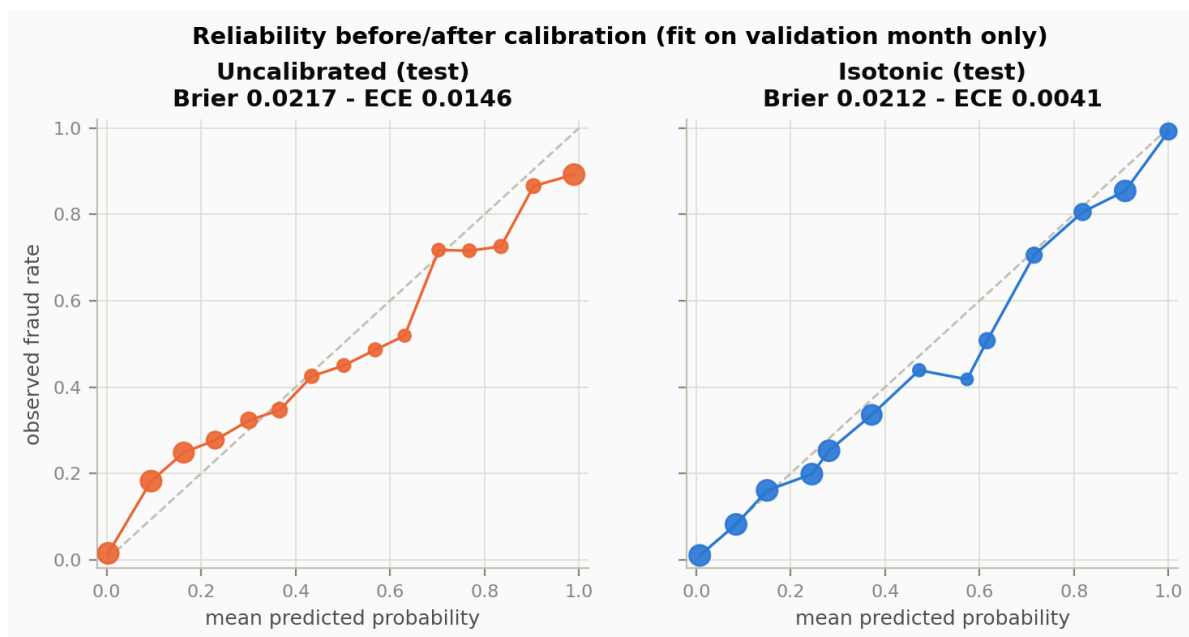


Figure 4 — before and after calibration: the dots move onto the diagonal, meaning 'says 30%' really is 30%

Why does this matter beyond tidiness? Because the *decision rule* in the next step is a statement about probabilities. An uncalibrated score feeds it garbage.

### The cost model — and what "k = 0.15: 39.4%" actually means

Here is the economic heart. For a purchase of amount **A**:

- Missing a fraud costs the full amount **A** (the write-off).
- Declining a real customer costs  $k \times A$  — a fraction covering the lost profit and the risk the customer walks away. The central assumption is **k = 0.15**: a false decline hurts 15% as much as a missed fraud of the same size.

A little algebra (shown in the paper) collapses everything into one rule: **decline whenever the calibrated fraud probability exceeds  $k/(1+k)$** . For  $k = 0.15$  that's 13%. Not 50% — because the two mistakes aren't equally priced, the rational bar for declining is low. And the rule doesn't depend on the amount: the amount appears on both sides of the trade-off and cancels out.

Performance is then measured in dollars as **savings**. On the final test month, approving everything would have lost **\$495,244** to fraud — that's the do-nothing baseline. The system's decisions produced a total bill (fraud it still missed, plus the cost of customers it wrongly declined) of **\$300,271**. It eliminated **39.4%** of the achievable loss — that is exactly what "savings at  $k = 0.15$ : 39.4%" means. The bracketed range next to it, [35.8%, 42.8%], is a **bootstrap confidence interval**: the test month is re-sampled a thousand times to ask "if the month's customers had been slightly different, how much would this number wobble?"

Two comparisons give the number teeth. Thresholds picked by popular accuracy formulas (F1, Youden's J) save only 31.4% and 27.4% on the same data — and if false declines get more expensive (higher  $k$ ), the Youden rule actually *loses* money, down to  $-238\%$ . And applying the cost rule to *uncalibrated* scores costs about **\$11,000 extra in one month** — calibration, priced in dollars.

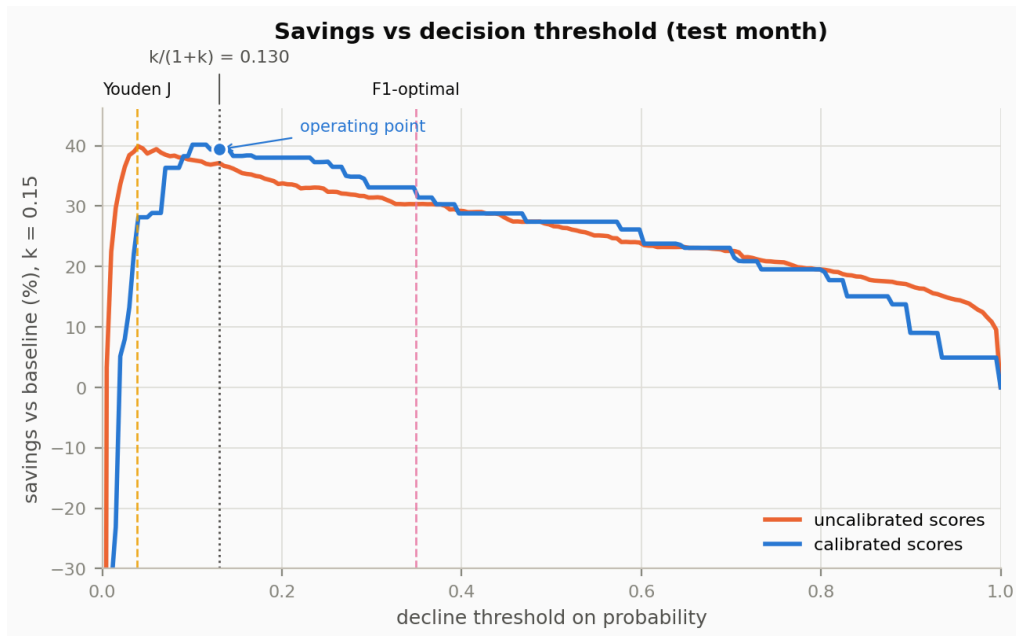


Figure 5 — savings depend on where you set the bar; the analytic threshold lands on the sweet spot

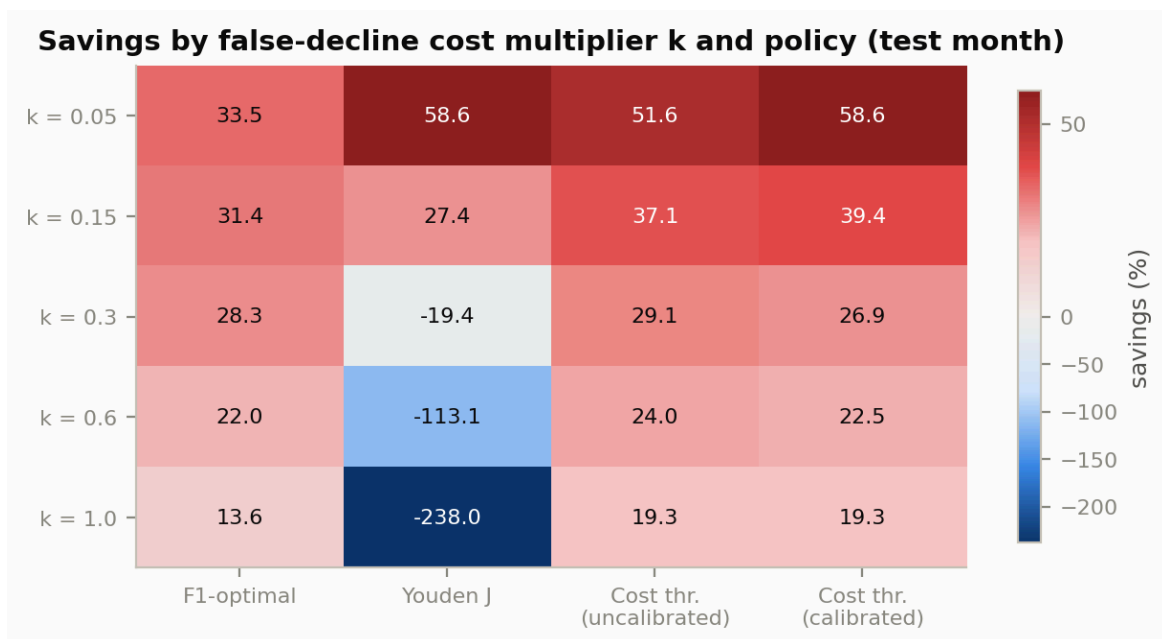


Figure 6 — stress-testing the assumption: savings across different values of  $k$

## Drift — why fraud models go stale

Fraud patterns move: attackers adapt, browsers update, shopping habits shift. **Drift** is that slow divergence between the world a model learned and the world it scores. Three maintenance policies were compared: never retrain (**static**), retrain monthly on all history (**expanding**), retrain monthly on a recent window (**sliding**). Result: one monthly retrain recovers about **7.5 points of savings** that the static model loses by month 5. Two monitoring tools quantify the drift itself. **PSI (population stability index)** asks, per input, "does this ingredient's mix still look like training?" — the biggest mover was the *browser version* field, which is exactly what you'd expect as software updates roll out. **Adversarial validation** is a spot-the-difference test: train a classifier to distinguish training-period rows from later rows; if it can (here it could, scoring 0.96 where 0.5 means "can't tell"), the data has measurably changed.

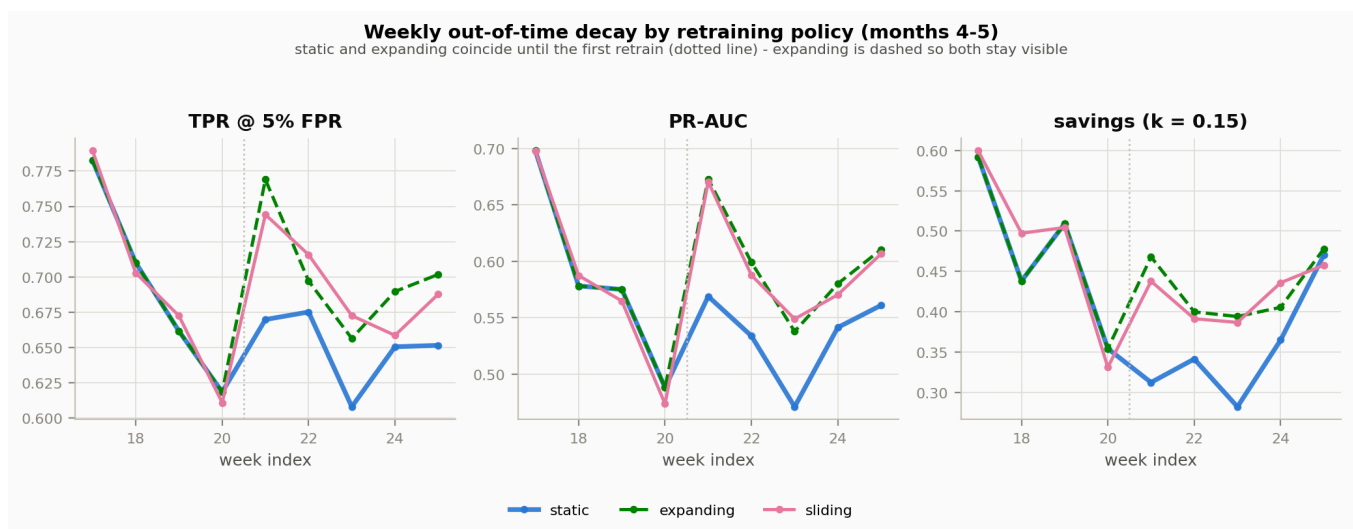


Figure 7 — weekly performance decay under three retraining policies

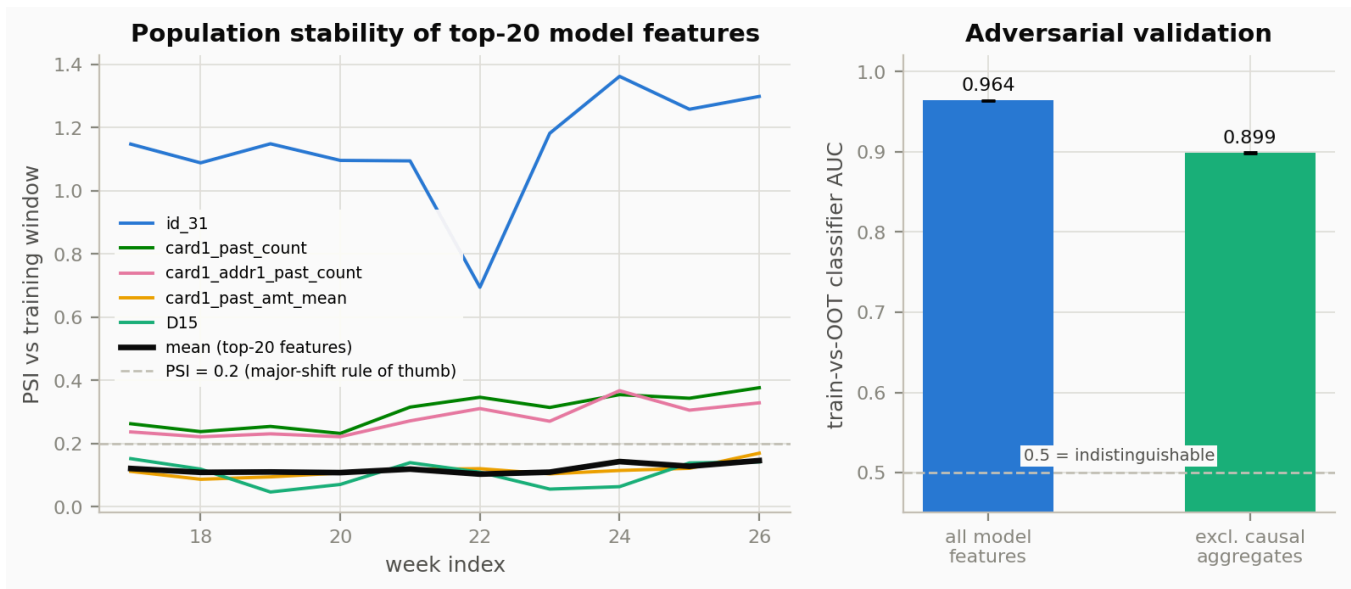


Figure 8 — where the drift lives: input stability and the spot-the-difference test

## Ablation — which data earns its keep

An **ablation** removes ingredients one at a time to see what each contributes. Feature groups were added cumulatively and removed one-by-one. Finding: the 40 most interpretable features (core transaction facts plus simple count signals) deliver **86% of the full system's savings**; the 339 anonymous engineered signals add about 2 points; the device/identity block adds a little detection but nothing in dollars at this operating point. For a regulated business this is a governance fact: most of the value is auditable, and every extra data feed has to justify its complexity.

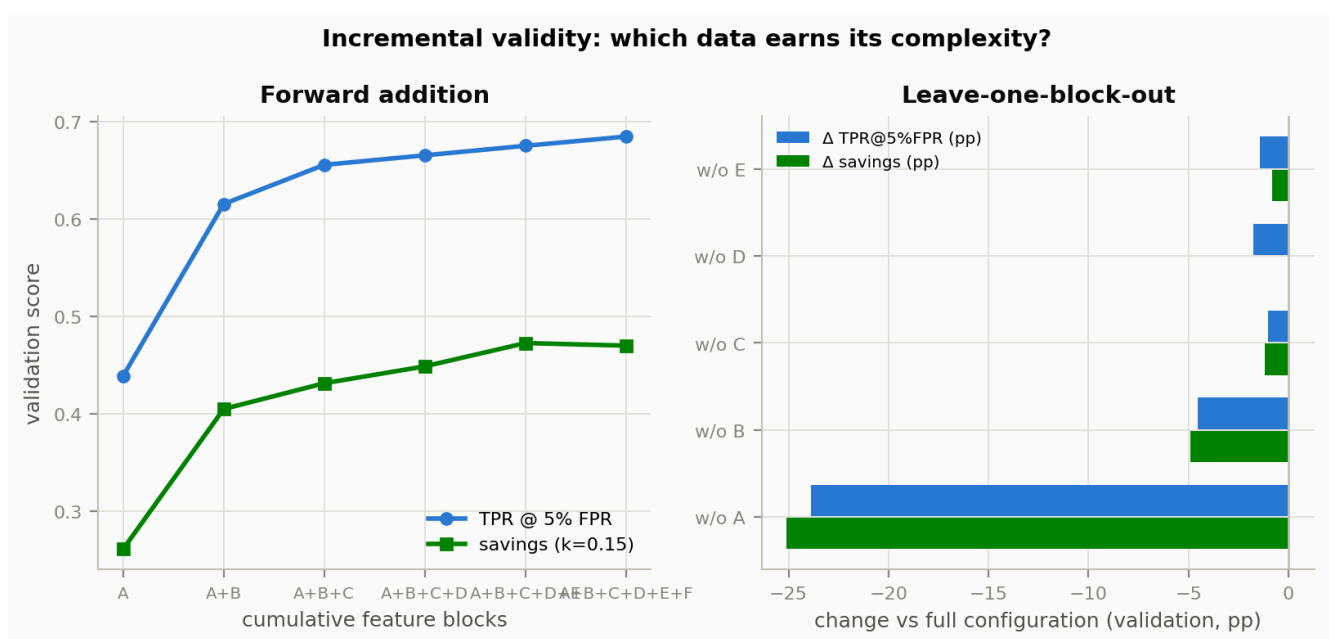


Figure 9 — value added by each block of features



## SHAP — itemized receipts for individual decisions

**SHAP** is a method (with game-theory roots) that splits one specific prediction into named contributions: *this* score is high because of +2.6 from an unusual count signal, +2.1 from this pattern, −0.9 from that reassuring one. It turns "the algorithm said no" into an itemized receipt — which is what a fraud analyst, an appeals process, or a regulator needs. The paper shows one correctly-caught fraud and one false alarm; instructively, the false alarm *is* a legitimate customer who happens to look brand-new, which is why mature systems route such cases to an extra verification step (a text-message check) instead of a hard decline.

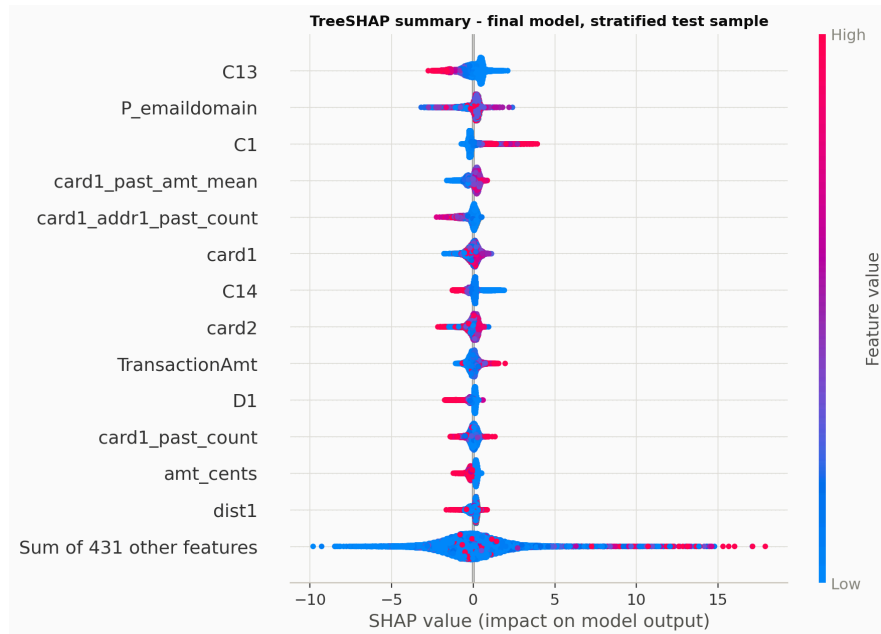


Figure 10a — the model's most influential signals across 20,000 test transactions

## The demo app and the AI-written narratives

A small interactive app accompanies the repo: adjust  $k$  and the threshold with sliders and watch costs move; inspect any transaction's receipt; monitor drift. One design principle worth noticing: the system can draft plain-language *investigation notes* for flagged transactions (via a large language model when a key is configured, via fixed templates otherwise) — but this sits on the **cold path**, helping a human afterwards. The real-time approve/decline decision stays with the deterministic, auditable model. That placement — LLMs assist investigation, never authorization — mirrors how payment networks describe their own AI governance.

## Glossary — quick reference

| Term                   | Plain meaning                                                                                               |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| Stage gate             | A pass/fail checkpoint between project phases; work stops until it passes                                   |
| EDA                    | Structured "look at the data first": volumes, gaps, trends                                                  |
| Leakage                | Future information contaminating training; makes lab results a lie                                          |
| Causal feature         | A signal computed only from events strictly before the transaction                                          |
| Baseline               | The simple alternative you must beat (here: approve-all, and a simple model)                                |
| Champion / challenger  | Simple trusted model vs. stronger candidate, compared on equal terms                                        |
| Cross-validation       | Testing on shuffled held-out slices; invalid for time-ordered fraud data                                    |
| Out-of-time (OOT) test | Testing strictly on a later period than training — the honest way                                           |
| Optimism gap           | How much the shuffled method overstates the honest result                                                   |
| ROC-AUC / PR-AUC       | Ranking-quality scores (1.0 = perfect); PR-AUC is stricter when fraud is rare                               |
| TPR @ 5% FPR           | With false alarms capped at 5%, the share of fraud caught (here 67.2%)                                      |
| Calibration            | Making "the model says 30%" mean "30% of these really are fraud"                                            |
| Platt / isotonic       | Two correction curves: one smooth S-shape; one flexible never-decreasing staircase                          |
| Brier score            | Accuracy-and-honesty combined; lower is better                                                              |
| ECE                    | Average gap between stated probability and observed reality                                                 |
| k                      | How bad a false decline is relative to a missed fraud (central case 0.15)                                   |
| $k/(1+k)$              | The mathematically best decline bar once scores are calibrated (13% at $k=0.15$ )                           |
| Savings                | Share of the do-nothing loss the system eliminates (headline: 39.4%)                                        |
| Bootstrap CI           | "Re-run the month 1,000 times on paper" wobble range for a number                                           |
| Drift                  | The world slowly stops resembling the training data                                                         |
| PSI                    | Per-input drift alarm; 0.2 is the conventional "major shift" line                                           |
| Adversarial validation | Classifier plays spot-the-difference between old and new data                                               |
| Ablation               | Remove ingredients to measure what each one contributes                                                     |
| SHAP                   | An itemized receipt for one specific model decision                                                         |
| Cold path              | Off-line, after-the-fact assistance (safe place for LLMs); the <i>hot path</i> makes the real-time decision |

*Companion produced alongside the main write-up; every number quoted here comes from the same audited result files.*